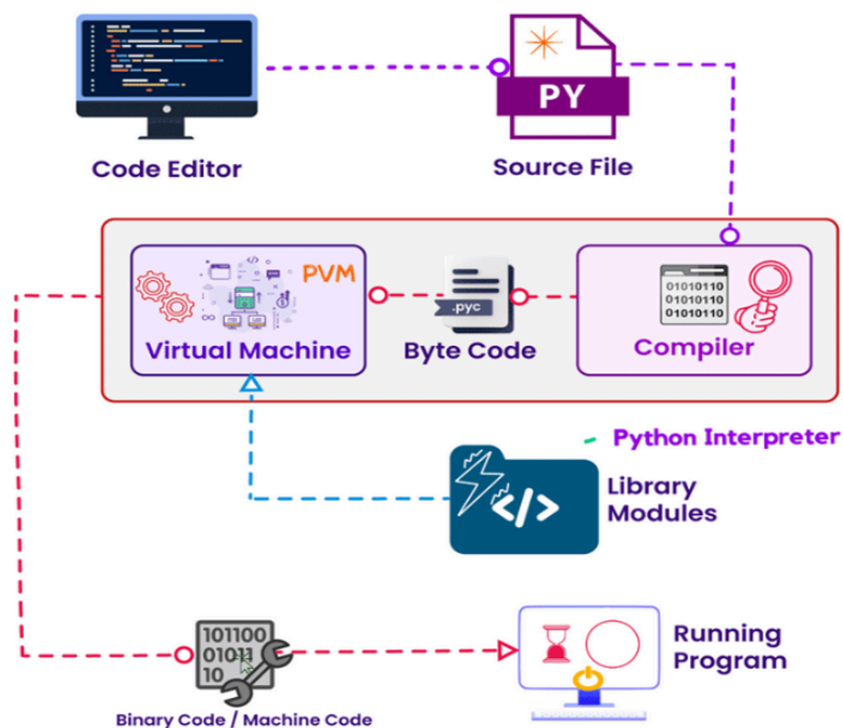
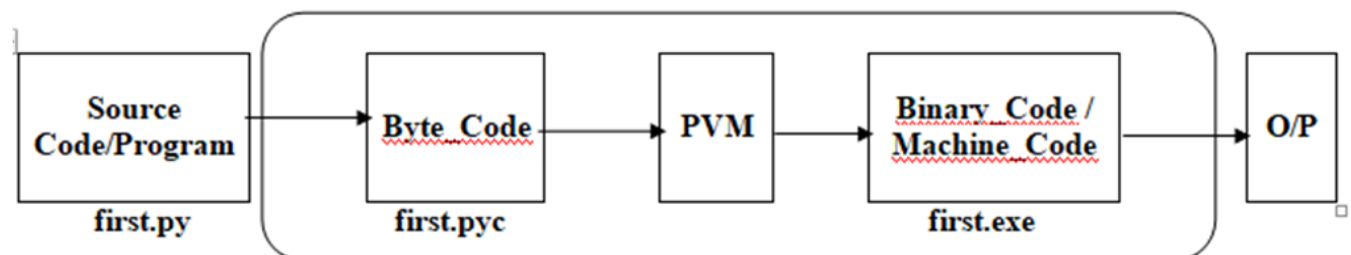


Internal Working of Python



Python can be described as a high-level, interpreted programming language with a clear syntax, making it user-friendly and widely used in many domains. Here's a breakdown of Each of these steps occurs behind the scenes, making Python a powerful and flexible language.



Examples:-

```
a = 10
b = 10
print("Sum ", (a+b))
```

The execution of the Python program involves 2 Steps:

Compilation

Interpreter

Compilation

The program is converted into byte code. Byte code is a fixed set of instructions that represent arithmetic, comparison, memory operations, etc. It can run on any operating system and hardware. The

byte code instructions are created in the .pyc file. The .pyc file is not explicitly created as Python handles it internally but it can be viewed with the following command:

```
PS E:\Python_data> python -m py_compile first.py
```

`-m` and `py_compile` represent module and module name respectively. This module is responsible to generate .pyc file. The compiler creates a directory named **pycache** where it stores the `first.cpython-310.pyc` file.

Interpreter

The next step involves converting the byte code (.pyc file) into machine code. This step is necessary as the computer can understand only machine code (binary code). Python Virtual Machine (PVM) first understands the operating system and processor in the computer and then converts it into machine code. Further, these machine code instructions are executed by processor and the results are displayed.

However, the interpreter inside the PVM translates the program line by line thereby consuming a lot of time. To overcome this, a compiler known as Just In Time (JIT) is added to PVM. JIT compiler improves the execution speed of the Python program. This compiler is not used in all Python environments like CPython which is standard Python software.

To execute the `first.cpython-310.pyc` we can use the following command:

```
PS E:\Python_data\__pycache__> python first1.cpython-310.pyc
```

view the byte code of the file – `first.py` we can type the following command as :

`first.py`:

```
x = 10
```

```
y = 10
```

```
z=x+y
```

```
print(z)
```

The command `python -m dis first.py` disassembles the Python bytecode generated from the source code in the file `first.py`.

`python`: This is the command to invoke the Python interpreter.

`-m dis`: This uses Python's built-in `dis` module to disassemble the Python bytecode. `dis` stands for disassembler. It translates Python bytecode back into a more readable form, showing the low-level instructions that the Python Virtual Machine (PVM) executes.

`first.py`: This is the Python script file whose bytecode will be disassembled.

When you run this command, Python compiles `first.py` into bytecode (if not already compiled), and the `dis` module disassembles it. This helps you understand the internal bytecode instructions that Python generates from your source code.

LOAD_CONST: Loads a constant value (like numbers 10 and 20).

STORE_NAME: Stores the value in a variable (like x, y, or z).

LOAD_NAME: Loads the value of a variable from memory.

BINARY_ADD: Adds two values (in this case, the values of x and y).

CALL_FUNCTION: Calls a function (like print).

RETURN_VALUE: Returns from a function (in this case, the main program).